

# Technical Assessment – Junior Software Engineer

---

## Overview

Build a small **AI-Assisted Task Review Application**.

The application will allow an operations user to view incoming tasks, update their status, and use AI to help understand what action should be taken.

We are mainly looking at your **programming fundamentals, learning ability, problem-solving approach, and code quality**.

---

## Technology

### Backend

Use either:

- JavaScript / TypeScript
- Python

You may use any suitable backend framework.

### Frontend

Preferably use **React**.

### Data Storage

You may use:

- In-memory storage
- JSON file
- SQLite

A production database is not required.

---

## Task

Each task should contain:

```
id
title
description
priority
status
```

createdAt

Statuses:

NEW  
IN\_PROGRESS  
COMPLETED

Priorities:

LOW  
MEDIUM  
HIGH

## 1. Task List

Create functionality that allows a user to:

- View tasks.
- Filter tasks by status.
- See task priority and creation date.

## 2. Update Task Status

Allow users to update a task status.

Example:

PATCH /tasks/:id/status

The backend should validate the submitted status and reject unsupported values.

## 3. AI Analysis

Add an **Analyse with AI** feature.

For example:

Title:  
Missing customer document

Description:

The customer submitted their application but has not provided their latest payslip.

The application should return something similar to:

```
{
  "category": "DOCUMENT_REQUEST",
  "priority": "HIGH",
  "summary": "Customer needs to provide their latest payslip.",
  "recommendedAction": "Request the missing payslip."
}
```

You may use any LLM provider.

If you do not have access to an LLM API, you may implement a mock AI service.

We mainly want to see that you understand how an application can:

Application -> Backend -> AI/LLM -> Structured Result -> User Interface

Your application should handle a basic AI/API failure without crashing.

## Suggested APIs

You may create APIs similar to:

GET /tasks

PATCH /tasks/:id/status

POST /tasks/:id/analyse

You may structure them differently if you prefer.

---

## Frontend

Create a simple interface that:

- Displays tasks.
- Filters tasks by status.
- Allows task status to be changed.
- Provides an Analyse with AI action.
- Displays the AI analysis.
- Shows appropriate loading and error states.

Visual design is not an important part of this assessment.

---

## Testing

Include at least **one meaningful automated test**.

For example:

- Valid task status is accepted.
- Invalid task status is rejected.
- AI failure is handled correctly.

You may choose which behaviour you would most like to test.

---

## AI-Assisted Development

You may use AI coding tools such as GitHub Copilot, Cursor, Claude Code, Codex, or similar tools.

We encourage you to use modern development tools, but you should understand and be able to explain the code you submit.

---

## README

Please include a short README covering:

- How to run the application.
  - Technologies you used.
  - Brief explanation of your approach.
  - What you would improve if you had more time.
  - Which AI coding tools you used, if any.
  - How you checked that AI-generated code was correct.
- 

## Submission

Please push your solution to a **GitHub repository** and share the repository link with us.

The repository may be **public or private**. If private, provide access to the reviewers specified by the recruitment team.

Please include:

- Source code
- README
- Test(s)
- Setup instructions

Do not commit API keys, passwords, or other credentials.

## Submission Format

Name:

GitHub Repository:

Backend Language:

LLM Provider / Mock Used:

Approximate Time Spent: